Nahom Solomon Email: nsolomon@horizon.csueastbay.edu
Christopher Ruiz-Guerra Email: cruizguerra@horizon.csueastbay.edu

12/5/2024

Lab 5: Configuring a VGA Oscilloscope

Introduction:

This laboratory experiment extends our prior work in developing a VGA controller module that enables graphical representations on a monitor using data sourced from an analog-to-digital converter (ADC). The ADC is linked to the field-programmable gate array (FPGA) board, where it converts analog voltage signals into digital signals. Once the conversion is complete, we store the resulting data as an array of digital values. These values are then transmitted as green pixels from the FPGA board through a VGA cable to the monitor, effectively generating a basic oscilloscope display.

This experiment highlights the skills we have cultivated throughout the semester. Our objective is to showcase our comprehension of the key concepts we have learned, building on our previously acquired knowledge. It emphasizes the significance of synchronization and the configuration of data output to achieve the desired results based on the parameters set for this experiment.

Theory:

The oscilloscope, together with the analog-to-digital converter (ADC), measures and converts electrical signals from analog to digital form. In this project, we are using an 8-bit parallel data converter, identified as AD7819. This ADC requires only a minimal number of signals to initiate a new conversion since most of its control logic is integrated within the circuit. The remaining control logic and clock functions are also embedded in the chip. The ADC operates at a sampling rate of 200 kilo samples per second (kS/s). Since the FPGA board

operates at a significantly higher speed, we incorporated delays in our Verilog code to provide the ADC with sufficient time to prepare for conversion.

The digital signals output by the ADC are labeled DB0 through DB7, representing the 8-bit value corresponding to the input voltage. These values become available only after the conversion is complete. The CONVSTb (Convert Start) signal initiates the analog-to-digital conversion process, while the BUSY signal indicates when the ADC is occupied, requiring the system to wait before collecting data. Additionally, the RDb (Read bit) and CSb (Chip Select bit) signals must be set to active low before reading the converted output value.

The required analog signals include Vin, which is the analog input voltage that is converted into a digital value. The input voltage can range from 0V to Vref, where Vref is the reference voltage or maximum voltage that can be read, corresponding to a maximum value of 255. The timing of the signals sent to the four digital control inputs must align with the timing parameters provided in the datasheet. The FPGA reads these data values from the ADC which then displays to the screen.

Experiment:

The first experiment focused on the timing and generation of the ADC clock signals, CONVSTb, CSb, and RDb, according to the datasheet. We began by establishing the timing for the ADC control signal. Since we are sampling at a rate of 200 kilo samples per second (kS/s), this will be our operating frequency, resulting in a period of 5,000 nanoseconds. With a clock speed of 10 ns on the Basys 3 board, we can achieve 500 cycles for the ADC converter.

The CONVSTb signal is initially set to logic low for 30 ns, equivalent to 3 clock cycles, and then rises until the next sampling period. The BUSY signal is generated by the ADC once

CONVSTb goes low, with a delay of 3 clock cycles, and it stays high for 4,500 ns or 450 clock cycles. When BUSY goes low, both the CSb and RDb signals are set to low, allowing us to sample data from the ADC converter after an additional clock cycle of delay.

We calculated the chip select period starting from the falling edge of CONVST to the data bus, which occurs 10 clock cycles before the end of our defined period. This results in an estimated chip select period of between 450 and 480 clock cycles for timing CSb. Additionally, the RDb signal rises 1 clock cycle before CSb. The sampling period indicates a 1-clock cycle delay on both ends of the RDb signal. Therefore, we begin sampling between 451 and 481 clock cycles to ensure a proper digital read from the ADC. To validate that the generated signal displays the correct output data, we created a TestBench to verify that the results meet the parameters provided in the datasheet. *See Figure 1.1 and Figure 1.2.*

In the second part of Experiment 1, we focused on assigning connections to the PMOD port pin on the FPGA board. We chose to connect the signals to the PMOD JA section. Specifically, CSb was connected to pin H1, RDb to pin K2, and CONVSTb to pin H2. The CLK was linked to the internal clock of the FPGA at pin W5, which operates at 100 MHz.

Using these pin assignments, we employed two probes to measure timing and verify that the timing delays were correct, matching the output from the TestBench. Additionally, we confirmed that we were receiving a frequency of 200 kHz and validated the pulse widths of each signal. *See Figure 1.3.*

In Experiment 2, we connected the ADC to a breadboard using jumper cables to link the FPGA's control signals and input data from the DB0 to DB7 pins to the PMOD JC ports.

This oscilloscope module generates synchronized VGA signals while interfacing with a CONVST for data capture. It operates on a 100 MHz system clock and derives a 25 MHz pixel clock (PCLK) using a 14-bit counter, essential for pixel updates and VGA timing synchronization.

The horizontal sync (hsync) signal, controlled by a counter (ctrH), defines each scanline's duration of 800 PCLK cycles. Each scanline consists of a 640-count visible area (25.6 µs), a front porch (16 counts), a pulse width (96 counts), and a back porch (48 counts). The vertical sync (VSYNC) spans 525 lines per frame, including 480 visible lines, 10 front porch lines, 2 pulse width, and 33 back porch lines. For ADC control, a timing_counter runs on the 100 MHz clock, defining a 500-count cycle. The CONVST signal remains low for the first 2 counts, with the chip select (CSb) active between counts 450 and 489, and the read (RDb) active between counts 450 and 479. Data is captured in a circular buffer of 640 entries when the ADC is ready, the timing_counter reaches 470 counts, and the switch is low.

The buffer counter, a 10-bit register, tracks data storage positions and resets after reaching 639 to enable continuous data capture. During VGA rendering, the buffer contents generate the green VGA signal, activating when the current vertical position (ctrV) matches the value stored in the buffer.

This design ensures synchronized VGA output, accurate ADC data acquisition, and smooth waveform rendering while adhering to VGA standards. The buffer counter is crucial for synchronizing data acquisition with VGA signal generation. *See Figure 2.1.*

Results:

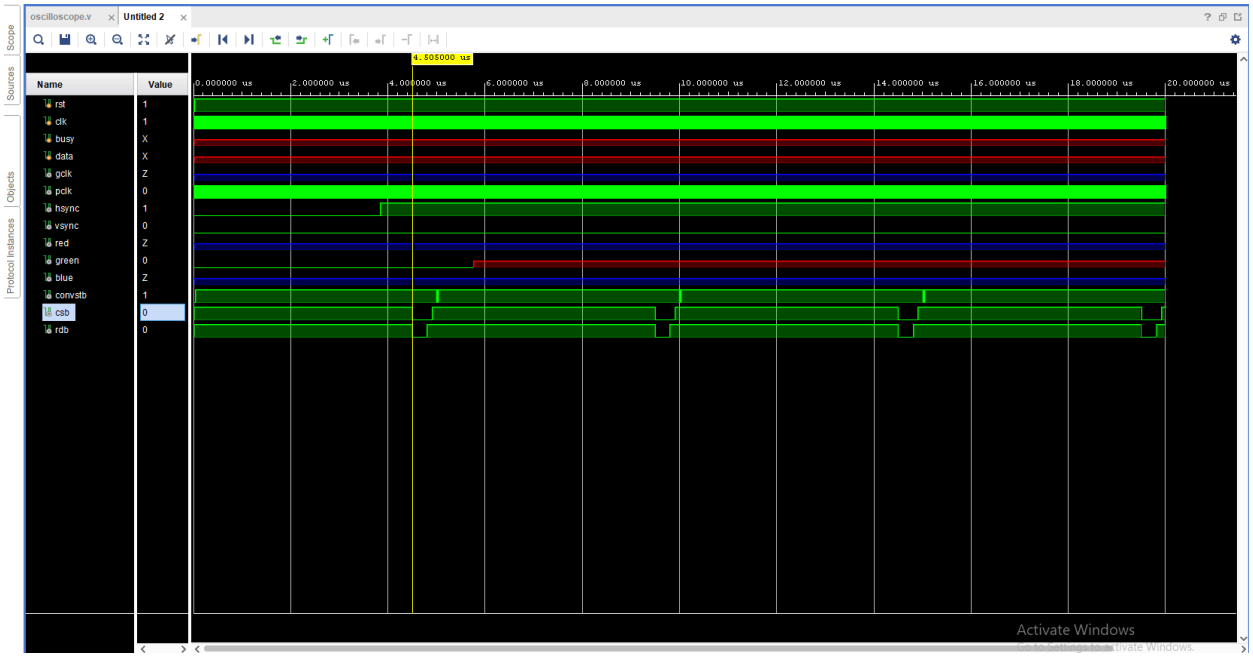

Figure 1.1

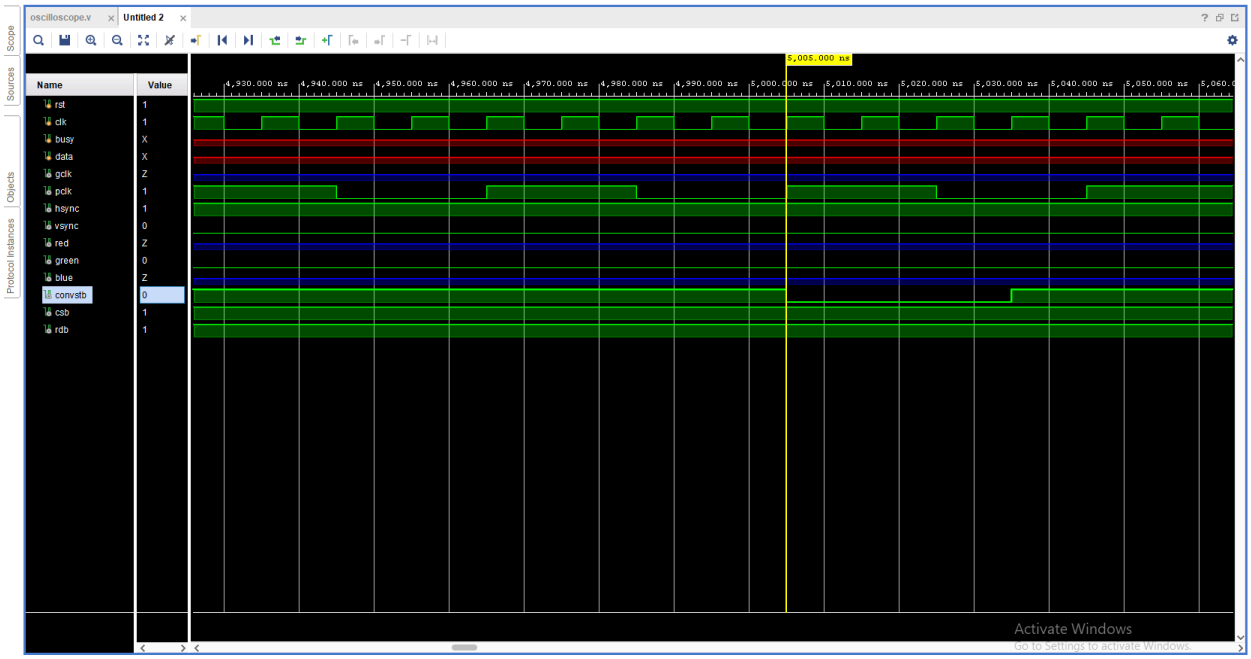

Figure 1.2

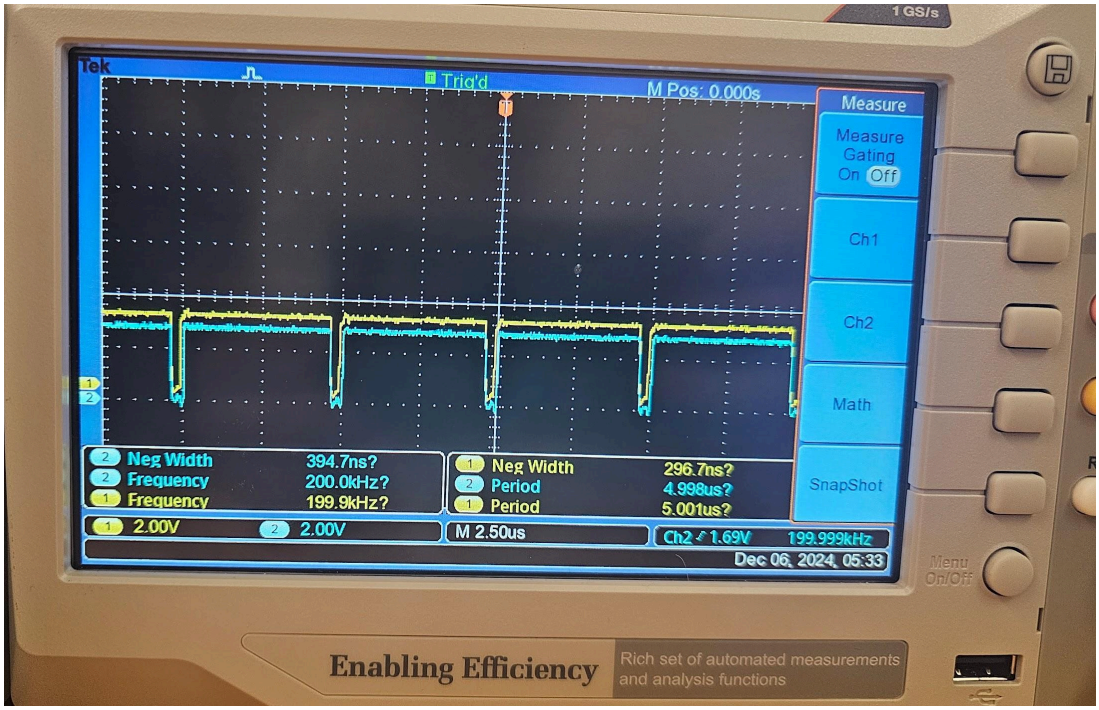
1 GS/s
Tek
Trig'd
M Pos: 0.000s
Measure
Measure Gating On Off
Ch1
Ch2
Math
SnapShot
Neg Width 394.7ns?
Frequency 200.0kHz?
Frequency 199.9kHz?
Neg Width 296.7ns?
Period 4.998us?
Period 5.001us?
2.00V
2.00V
M 2.50us
Ch2 1.69V 199.999kHz
Dec 06, 2024, 05:33
Menu On/Off
Enabling Efficiency
Rich set of automated measurements and analysis functions

Figure 1.3

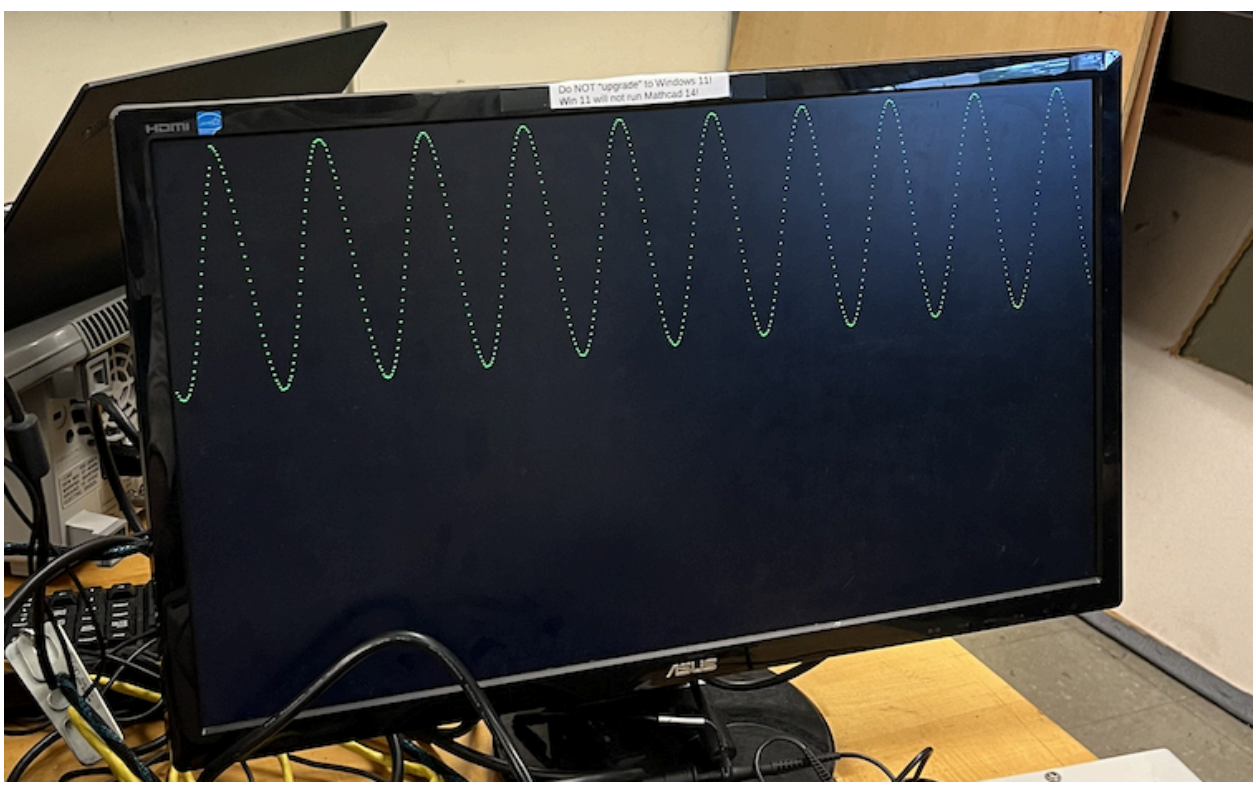
Do NOT "upgrade" to Windows 11!
Win 11 will not run Mathcad 14!
HDMI
ASUS

Figure 2.1

Discussion:

The results for Figures 1.1 and 1.2 are aspected, there was a small amount of noise from the oscilloscope from Figure 1.3 that verifies our signal. The noise can be attributed to the amount wires as they were fairly close to each other in addition to other noise from neighboring groups with their power supplies. The results are the same as our testbench and were marked off, thus no discrepancies. Figure 1.3 also had no discrepancies, even though our waves may not be "perfect" This is due to the sampling that we're taking and the timing of the ADC.

Conclusion:

In conclusion, this oscilloscope module effectively generates VGA signals and shows waveforms in real-time. It keeps timing consistent using counters, which helps it meet VGA standards while capturing data from the analog-to-digital converter (ADC) efficiently. The design uses modules and a buffer counter for storing and retrieving data, ensuring smooth integration between data collection and display.

From our experience, coordinating horizontal and vertical sync signals with the captured data allows for clear and accurate waveform displays. This design highlights the importance of understanding timing and how hardware signals interact to create functional systems.

With these ideas in mind, this implementation provides a strong foundation for building more advanced real-time visualization systems. It shows how hardware logic can connect digital processing with what users see.